

The Cost of Centralised Leadership in Distributed Consensus

An Implementation-Level Measurement Study of Leader-Based and Leaderless Consensus

Sephy Ryu
Independent Researcher
South Korea

Abstract

Leader-based consensus protocols such as Raft route normal write requests through an elected leader, whereas leaderless protocols such as EPaxos permit proposal initiation at multiple replicas. This paper presents an implementation-level experimental comparison of these two approaches under a common benchmark methodology: the same workload generator, CPU and memory allocations, failure injection, and measurement procedures, with ten independent repetitions across seven experiment families and replica counts from three to nine on a single containerised host.

In the primary implementations, EPaxos achieved higher median throughput than Raft at three and five replicas (ratios 1.22 and 1.73, exact Mann-Whitney $p < 0.001$), but the difference was not significant at seven or nine replicas (ratios 1.18 and 0.99, $p = 0.12$ and $p = 0.32$). Eight of 1,302 repetitions stalled (two Raft, six EPaxos), so we report medians, which are robust to these stalls, rather than means. The EPaxos throughput decline at seven and nine replicas is associated with request timeouts: every EPaxos run at those sizes except one had requests time out at the 2,000 ms client timeout, each occupying a client worker for the full 2s, while the successful requests themselves completed in 5.4–5.6 ms, the same as at three replicas. Median latency was flat for EPaxos (5.4–5.6 ms) across replica counts while Raft grew from 6.9 to 14.6 ms. Client concurrency produced a larger separation at high concurrency: at concurrency 256, EPaxos measured 41,077 req/s against 19,333 req/s for Raft, with p50 latency 5.6 ms against 12.1 ms. The read/write-ratio and conflict-ratio experiments showed no clear systematic throughput trend, and EPaxos used approximately $1.8\times$ the peak memory of Raft.

Failure experiments revealed a more specific distinction in request-path dependence. Losing the Raft leader produced availability gaps of 1.81–4.74 s and thousands of failed requests, whereas losing a Raft follower or an EPaxos replica produced substantially shorter interruptions in the tested three-replica configuration.

To examine implementation sensitivity, the study additionally evaluates two Raft implementations and two EPaxos implementations. Their responses to application-level one-way communication delay differed substantially: at 10 ms of injected cost, throughput decreased by 72–96 % across the implementations, while the etcd Raft implementation decreased by 94 % at 1 ms. The delay also changed the ordering: from 1 ms onward, HashiCorp Raft sustained higher throughput than the original EPaxos implementation (and, at 5–10 ms, than nvb EPaxos as well), so the leader-based/leaderless

ranking is not invariant to communication conditions. A separate correctness harness performed 18 deterministic state-convergence tests covering concurrency levels of 1, 8, and 32 and the tested replica-failure scenarios; all tests passed. The results should therefore be interpreted as controlled measurements of the evaluated implementations rather than universal properties of Raft or EPaxos.

Keywords

consensus, Raft, EPaxos, leader-based ordering, leaderless ordering, benchmarking

1 Introduction

Distributed systems often replicate state across multiple machines to provide fault tolerance and continued service in the presence of failures. When a replicated state machine processes an operation, its replicas must coordinate to maintain a consistent state despite concurrent execution and failures. Consensus protocols provide the coordination mechanisms required for this process and are therefore fundamental to many distributed systems.

Paxos is one of the foundational consensus algorithms for reaching agreement among distributed nodes in the presence of failures [4]. Although its formulation is influential, the original presentation is highly abstract and leaves substantial implementation decisions to system designers. Consequently, systems implementing consensus can differ considerably in their execution paths, storage mechanisms, communication patterns, and resource requirements even when they implement the same underlying protocol family.

Raft was introduced with an emphasis on understandability and implementability while providing fault-tolerant consensus [6]. A central feature of Raft is its explicit leader. Client operations are submitted through the leader, which determines the order of log entries and coordinates their replication before commitment. This leader-based structure provides a clear separation between request coordination and replica execution. It also creates a characteristic coordination pattern: normal request processing is concentrated at the leader, and leader failure requires an election before leader-based request processing can resume.

EPaxos provides a contrasting leaderless approach [5]. Rather than relying on a single replica to determine a global order for all operations, EPaxos allows replicas to propose operations and determine dependencies among them. Operations that do not conflict can therefore avoid some of the coordination associated with a single global ordering point.

This structure does not eliminate consensus overhead, however. Dependency tracking, conflict detection, and additional coordination can introduce their own communication and computational costs.

These architectural differences motivate an empirical comparison, but they also make a simple protocol-level performance ranking difficult to interpret. A protocol may perform differently as the number of replicas, client concurrency, operation conflicts, or communication delay changes. Furthermore, measured performance is affected by implementation-level mechanisms such as persistence, scheduling, message handling, and resource management. As a result, a performance difference observed between two implementations cannot automatically be attributed to the consensus algorithm alone.

This distinction is particularly important when comparing leader-based and leaderless consensus. The presence of a leader provides a structural difference in how requests are coordinated, but it does not by itself determine the measured throughput or latency of a concrete implementation. Conversely, an observed performance difference between two implementations may arise from the interaction between protocol structure and implementation mechanisms. An experimental comparison should therefore examine both absolute performance and how that performance changes across controlled conditions.

In this paper, we experimentally compare Raft and EPaxos as representative leader-based and leaderless consensus protocols. We evaluate throughput, latency, resource utilisation, and failure behaviour under controlled variations in replica count, client concurrency, operation conflict rate, read/write composition, and communication conditions. We additionally compare multiple implementations to examine how implementation choices affect the measured relationship between the two protocol families.

We focus on the following research questions:

- **RQ1:** How do the measured throughput and latency of the evaluated leader-based and leaderless implementations change with replica count, client concurrency, conflict rate, and read/write composition?
- **RQ2:** How do the evaluated implementations behave under replica and leader failures, particularly with respect to availability gaps and request failures?
- **RQ3:** How do communication conditions and implementation choices affect the measured performance differences between Raft and EPaxos?

We make the following contributions:

- **Reproducible benchmark infrastructure:** We build the Consensus Lab, which executes Raft and EPaxos in a common containerised environment through a common client harness and benchmark-facing interface, providing a controlled basis for measuring their behaviour.
- **Sensitivity-oriented evaluation:** We measure throughput and latency across controlled variations in replica count, client concurrency, conflict rate, read/write

composition, and communication conditions rather than relying on a single benchmark configuration.

- **Failure behaviour analysis:** We characterise request processing and availability under Raft leader failure, Raft follower failure, and EPaxos replica failure, separating leader-election effects from replica-failure observations.
- **Implementation-level analysis:** We evaluate multiple implementations of the two protocol families in selected experiments to quantify the extent to which implementation choices affect the measured performance relationship.
- **Open infrastructure:** We release the lab, experiment configurations, and analysis pipeline at <https://github.com/Sephy314/Leader-consensus-comparison>, allowing the experimental procedure and processed results to be independently re-analysed.

The experiments use a common containerised execution environment and the same client workload model for the evaluated implementations. Performance measurements are based on repeated runs across controlled configurations, with raw per-run observations retained for independent analysis. Because the experiments are conducted in a local environment rather than across physically distributed machines, the results are measurements of the evaluated implementations under the specified experimental conditions and should not be interpreted as universal production-performance estimates.

The remainder of this paper is organised as follows. Section 2 provides background on distributed consensus and discusses leader-based and leaderless approaches together with related work. Section 3 describes the Consensus Lab, the implementations under test, the client harness and workload model, the metrics, and the experiment suite. Section 4 presents the experimental results on throughput, scalability, latency, resource utilisation, failure behaviour, communication conditions, and implementation differences. Section 5 interprets the results with respect to the research questions and discusses the observed trade-offs between the evaluated implementations. Section 6 discusses threats to the validity of the measurements. Section 7 summarises the findings.

2 Background and Related Work

2.1 Distributed Consensus

Distributed consensus is the problem of enabling multiple distributed processes to agree on a common value or sequence of values despite failures and unreliable communication. In a replicated state machine, this agreement is particularly important because replicas must apply the same sequence of operations in order to maintain a consistent state. Without an agreement mechanism, different replicas may independently accept or apply different operations, causing their states to diverge.

Consensus therefore provides a fundamental abstraction for building fault-tolerant distributed systems. A consensus protocol must maintain safety, ensuring that correct replicas do not make contradictory decisions, while also providing

liveness under the conditions assumed by the protocol. For replicated state machines, these properties are closely connected to the problem of establishing a consistent ordering of state-changing operations.

Consensus protocols can be broadly distinguished by how they coordinate this ordering. One major approach uses a distinguished leader that coordinates the replication of operations, while another distributes the coordination process among multiple replicas without relying on a single permanent ordering authority. Raft represents the former approach, while EPaxos represents the latter.

2.2 Leader-Based Consensus: From Paxos to Raft

Paxos is one of the foundational protocols for distributed consensus. It provides a formal approach to reaching agreement among distributed processes in the presence of failures. However, the original presentation of Paxos is highly abstract, and constructing a complete replicated state machine requires additional mechanisms and implementation decisions beyond the core consensus abstraction [4].

Raft was designed as an alternative consensus algorithm with understandability as a primary design goal. Rather than presenting consensus as a single abstract problem, Raft decomposes it into relatively independent components, including leader election, log replication, and safety [6]. This decomposition provides a more explicit structure for reasoning about how a replicated state machine reaches agreement and maintains correctness.

A central feature of Raft is its leader. At any given term, one replica acts as the leader while the remaining replicas act as followers. Client requests that modify replicated state are normally directed to the leader. The leader appends these operations to its log and replicates the corresponding entries to follower replicas. Once an entry has been replicated to a sufficient quorum, it can be committed and subsequently applied to the replicated state machines [6].

This architecture simplifies the ordering problem by establishing a clear ordering authority. Rather than requiring replicas to independently determine the ordering of every write, followers primarily replicate the log produced by the leader. The Raft design also defines safety properties and mechanisms for preserving them during log replication and changes in leadership [6].

However, the same structure introduces a dependency on the leader. Since the leader is the primary entry point for replicated writes, the write path is structurally concentrated around a single replica. Adding additional follower replicas therefore does not directly provide additional independent write-ordering capacity. Under sufficiently high write workloads, the leader may become a bottleneck.

The leader also introduces a failure mode that does not exist in the same form in a leaderless protocol. When the current leader becomes unavailable, the remaining replicas must elect a new leader before normal leader-based processing can resume. During this transition, new writes cannot

be committed through the normal leader-based path. The election process therefore introduces a period in which failure of the ordering authority can affect request processing.

Thus, Raft provides a clear and structured approach to consensus by centralising the ordering of replicated operations around a leader. This design can simplify coordination and reasoning about correctness, while also creating dependence on the processing capacity and availability of the leader.

2.3 Leaderless Consensus: EPaxos

Leaderless consensus takes a different approach to distributed ordering. Instead of requiring all operations to pass through a single distinguished leader, multiple replicas can participate in proposing operations and determining their dependencies. EPaxos was designed around this approach, motivated in part by the limitations of leader-based consensus when a single replica becomes a central point for coordinating operations [5].

In EPaxos, a client can send a command to any replica, which can act as the replica responsible for proposing that command. This removes the requirement for a single replica to serve as the universal ordering point and allows operations to be proposed concurrently at different replicas [5].

Rather than imposing a single global ordering on all operations through a leader, EPaxos establishes dependencies between commands. Commands that are independent can potentially be committed without requiring them to be ordered relative to one another, while conflicting commands require their dependency relationships to be established consistently [5].

This design allows EPaxos to exploit command independence. When operations do not conflict, replicas can process proposals concurrently instead of serialising all operations through one leader. The resulting degree of parallelism depends on the workload and, in particular, on the extent to which concurrently proposed commands are independent [5].

Leaderless operation also changes the system's behaviour under replica failures. The failure of a particular replica does not necessarily require the remaining replicas to elect a new global leader before they can continue proposing commands. As long as the protocol can obtain the required quorum and satisfy its communication assumptions, other replicas can continue participating in the consensus process [5].

However, removing the single ordering authority does not eliminate coordination. Because multiple replicas can propose commands concurrently, EPaxos must determine the dependencies between commands and ensure that replicas execute conflicting operations in a compatible order. These mechanisms introduce additional coordination and computation compared with a system in which a single leader directly determines the order of operations [5].

The resulting trade-off is therefore not simply between consistency and availability. Instead, leader-based and leaderless consensus place coordination costs in different parts of the system. Leader-based consensus concentrates ordering around

a leader, while leaderless consensus distributes the responsibility for ordering across replicas and relies on dependency management to coordinate concurrent operations.

2.4 Related Work

Prior research has examined both the correctness and practical performance of consensus protocols. Paxos established a foundational framework for distributed agreement, while Raft provided a more explicitly structured leader-based approach with an emphasis on understandability [4, 6]. EPaxos subsequently explored a leaderless approach that exploits command independence and dependency relationships to reduce the reliance on a single ordering authority [5].

Comparative studies of consensus designs have reached different conclusions depending on the systems and workloads examined. Howard and Mortier compared Paxos and Raft implementations and found that the choice of implementation and configuration often matters more than the protocol family itself [3]. van Renesse et al. compared Paxos with leaderless replication and showed that the relative performance of leader-based and leaderless approaches depends on the workload, particularly on the degree of command conflict [9]. These results motivate evaluating multiple independent implementations of each protocol family, as we do here.

The practical evaluation of EPaxos has also been revisited. Tollman et al. re-evaluated EPaxos under geo-replicated workloads and found that the original benchmarking approach did not fully expose the performance impact of conflicting commands. Their results confirmed the low median commit latency reported for EPaxos in WAN settings while observing substantially worse tail latency, with performance being particularly sensitive to application workload [7]. This work illustrates that empirical conclusions about consensus protocols can depend not only on the protocol design itself, but also on workload characteristics and experimental methodology.

Our study follows this experimental perspective while examining a different dimension of consensus behaviour: the effect of centralised versus distributed ordering under a common local experimental environment. We compare two Raft implementations (HashiCorp Raft and etcd Raft) with two EPaxos implementations (the original `efficient/epaxos` and `nvanbenschoten/epaxos`) while varying replica count, client concurrency, communication delay, and failure conditions. The comparison therefore evaluates concrete implementations under a controlled workload rather than treating the results as protocol-independent properties of Raft or EPaxos.

The distinction is important when interpreting the measured performance differences. The Raft paper characterises Raft’s strong leadership as a central structural feature of the protocol, whereas EPaxos allows commands to be proposed at different replicas and exploits command independence to avoid relying on a single ordering authority [5, 6]. Our experiments investigate how these architectural differences manifest in throughput, latency, resource utilisation, and recovery behaviour under the evaluated workload, and how much of

the observed difference is attributable to the implementation rather than to the protocol structure.

Accordingly, the results should be understood as evidence about the evaluated protocol implementations and experimental conditions, rather than as universal performance characteristics of all Raft and EPaxos implementations. This distinction is also reflected in the threats to validity discussed in Section 6.

3 Experimental Setup

3.1 Consensus Lab Architecture

The experiments are conducted with the Consensus Lab, a reproducible benchmark infrastructure that runs the evaluated implementations in a common containerised environment. Each run is instantiated as an independent Docker Compose project containing one coordination service, one container per replica, and one benchmark client. The coordination service is the upstream EPaxos master [5] for EPaxos or a lab-provided Raft master that exposes the corresponding benchmark-facing RPC surface. All containers use the same runtime image and the configured CPU and memory allocations.

Both protocols expose the same benchmark-facing request interface based on the upstream EPaxos `generic smr` wire protocol (`PROPOSE` $\rightarrow$ `ProposeReplyTS`). A single client binary and workload generator are used for both protocols, so the operation mix, key-selection process, concurrency control, and request-generation logic are shared. Request routing differs according to the protocol architecture: Raft requests are directed to the current leader reported by HashiCorp Raft, whereas EPaxos requests are distributed round-robin across replicas. Reads are submitted through the consensus path in both protocols; no Raft-specific local-read optimisation is enabled.

3.2 Implementation

Each protocol is represented by two independent implementations (Table 1). The lab adapters add only orchestration, monitoring, and the communication-delay injection described below; they never implement or modify consensus logic.

Raft is provided by `github.com/hashicorp/raft` v1.7.3 [2] and wrapped in a thin adapter (`lab/adapters/raft/`). The adapter converts benchmark requests into state-machine commands, invokes `raft.Raft.Apply`, waits for the resulting future, and returns the resulting response. Leader election, log replication, quorum calculation, commitment, term management, and Raft state transitions are performed by the HashiCorp Raft library rather than by the adapter. Raft replicas use BoltDB-backed persistent state stored in per-replica Docker volumes, with a snapshot threshold of 8,192 entries and 1,024 trailing logs.

The second Raft implementation uses `go.etcd.io/raft` v3.7.0 [1], wrapped in a lab adapter (`lab/adapters/etcdraft/`) that follows the library’s asynchronous-storage-writes interface: a dedicated append thread batches up to 64 log entries into one write-ahead-log write with a single `fsync`, mirroring HashiCorp Raft’s group commit. The adapter

sets `MaxSizePerMsg=4,096` and `MaxInflightMsgs=256` (the library provides no defaults), enables pre-vote to match HashiCorp Raft, and compacts the log below `applied - 1,024` trailing entries. Raft replicas persist their log to a write-ahead log in per-replica Docker volumes.

EPaxos uses the upstream `efficient/epaxos` server and master [5], pinned at commit `791b115` and executed with `-e -exec -dreply`. EPaxos replica state is maintained in memory under the upstream configuration. The upstream implementation fixes the dependency-set size at five; the lab extends this field to support up to nine replicas through a documented wire-format extension. The extension changes the representable dependency-set size but does not alter the protocol phases, quorums, or decision rules. The same patched binary is used by all EPaxos replicas.

The second EPaxos implementation uses `nvanbenshoten/epaxos` [8], pinned at commit `425bd36` and wrapped in a lab adapter (`lab/adapters/nvbepaxos/`) that bridges the library’s gRPC peer transport to the benchmark’s client protocol. EPaxos replica state is in-memory in both implementations.

3.2.1 Communication-delay injection. The communication-cost family adds a fixed one-way delay to every inter-replica message. The injection is implemented in each implementation’s send path, so it exercises the same message flow as the baseline runs:

- **HashiCorp Raft:** the adapter’s transport wrapper (`isolatingTransport`) sleeps before forwarding `AppendEntries`, `AppendEntriesPipeline`, `RequestVote`, and `RequestPreVote`.
- **etcd Raft:** each per-peer sender goroutine sleeps before writing its message frame.
- **EPaxos (original):** the upstream `generic smr` send path sleeps before writing each outbound message. This is the one place the lab modifies the upstream EPaxos binary beyond orchestration; the change is confined to the send path and does not touch protocol logic.
- **EPaxos (nvb):** the adapter’s per-peer sender goroutine sleeps before each batch.

The delay is applied per message, so an implementation that sends more messages per request, or serialises its sends, is affected more strongly. In particular, the etcd Raft adapter’s per-peer sender sleeps on the adapter’s critical path: a single slow peer stalls that peer’s message stream, and because the delay is applied per message rather than per batch, the effective serialisation cost grows with the number of messages. The communication-cost results in Section 4 should therefore be read as measurements of each implementation’s send-path structure under added delay, not as a protocol-level latency-sensitivity ranking.

Because the implementations use their native storage and execution mechanisms, the measurements evaluate complete implementations rather than isolated consensus kernels. Consequently, observed performance differences include the effects of implementation mechanisms such as persistence, message processing, scheduling, and state-machine execution.

Table 1: The four evaluated implementations. Each protocol is represented by two independent implementations; the lab adapters add only orchestration, monitoring, and the communication-delay injection described in Section 3, never consensus logic.

Implementation	Consensus core	Storage	Delay inject
Raft (HashiCorp)	<code>hashicorp/raft</code> v1.7.3	BoltDB (persistent)	transport w
Raft (etcd)	<code>go.etcd.io/raft</code> v3.7.0	WAL (persistent, batched fsync)	per-peer sleep
EPaxos (original)	<code>efficient/epaxos</code> @ 791b115	in-memory	send-path s
EPaxos (nvb)	<code>nvanbenshoten/epaxos</code> @ 425bd36	in-memory	per-peer sleep

3.3 Client Harness and Workload Model

The benchmark client generates requests over a keyspace of 1,000 keys with a configurable read/write composition. The write ratio denotes the target fraction of generated requests that are writes; the remaining requests are reads. The client uses a closed-loop workload model with one worker per unit of configured concurrency. A worker issues its next request only after its previous request has completed, so the configured concurrency represents the maximum number of in-flight requests rather than a target arrival rate.

Both protocols receive requests generated by the same workload process. For conflict experiments, a configured conflict ratio determines the probability that a request selects from a shared hot-key set. Requests that do not select a hot key are drawn uniformly from a disjoint cold-key range. The realised hot-key fraction is therefore measured from the generated request sequence rather than assumed to equal the configured probability. Workload seeds are recorded for every repetition.

Each run begins with a warm-up phase followed by a measured phase. Requests issued during warm-up are not included in performance measurements.

3.4 Metrics

For each request, the client records the run identifier, protocol, replica count, workload parameters, concurrency, operation type, key, target replica, start and completion timestamps, latency, and success or failure status. The analysis pipeline derives throughput, success rate, and p50, p95, and p99 latency from these observations.

Per-replica resource telemetry is collected at approximately 200 ms resolution. CPU usage is obtained from `cgroup v2 cpu.stat`, network RX/TX counters from `/proc/<pid>/net/dev` within the relevant container network namespace, and resident memory from `/proc/<pid>/status`. Raw samples are retained separately for each replica rather than aggregated at collection time, allowing resource distribution and workload concentration across replicas to be examined.

3.5 Testbed Configuration and Parameters

Each performance run consists of a 3-second warm-up followed by a 20-second measured phase and is repeated independently ten times. Each replica container is allocated 2.0 CPUs and 512 MB of memory. The client is allocated 2.0 CPUs and 512 MB, while the coordination service is allocated 1.0 CPU and 256 MB. GOMAXPROCS is set to 4 for all replicas. The request timeout is 2,000 ms. Raft uses a 1,000 ms heartbeat timeout, a 2,000 ms election timeout, a snapshot threshold of 8,192 entries, and 1,024 trailing log entries.

The primary benchmark configuration uses the implementations’ configured storage mechanisms. Raft therefore retains its persistent state during a run, while EPaxos uses its upstream in-memory state model. This difference is part of the implementation configuration being measured rather than being removed from the performance path. Docker volumes are nevertheless isolated between repetitions so that no consensus state is carried from one repetition to another.

To examine sensitivity to communication delay, the benchmark additionally supports application-level one-way message-delay injection. For selected experiments, each outbound inter-replica message is delayed before being written to its socket. A configured delay cost c is combined with an optional uniformly distributed jitter term:

$$d = c + U\left(0, \frac{cj}{100}\right),$$

where c is specified in milliseconds and j is the jitter percentage. The evaluated costs are 0, 1, 3, 5, and 10 ms. The jitter sweep uses 0, 5, 10, 50, and 100 % at the 5 ms cost level.

This mechanism models additional application-level message delay rather than physical network latency and is therefore interpreted as a controlled communication-condition experiment. In particular, it does not claim to reproduce all effects of a physical network, such as queueing, packet loss, or link-level scheduling. The 0 ms configuration is the local-network baseline used by the other experiment families.

Each repetition uses a fresh Docker Compose project with new containers, networks, and storage volumes. Before execution, the harness checks that the required environment is free of leftover containers, networks, volumes, and occupied ports. After execution, the teardown is verified. Raft’s persistent state, including its log, term and voting state, snapshots, and cluster metadata, is therefore isolated to a single repetition and cannot affect subsequent repetitions. A run that fails the isolation checks is recorded as an invalid attempt rather than silently reused.

To reduce systematic effects from execution order, configurations are scheduled using blocked randomisation. Within each repetition block, every configuration in the corresponding experiment family appears exactly once in a shuffled order generated from a recorded seed. Each repetition also uses an independent workload seed derived from the base seed and repetition index. Both schedule and workload seeds are recorded in run metadata.

The principal experiment dimensions are replica count (3, 5, 7, and 9), write ratio (0, 10, 50, 90, and 100 %), conflict ratio (0, 10, 25, 50, 75, 90, and 100 %), client concurrency (1, 2, 4, 8, 16, 32, 64, 128, and 256), and application-level communication cost (0, 1, 3, 5, and 10 ms).

3.6 Experiment Suite

The experiment suite contains seven experiment families. Unless otherwise specified, each configuration is evaluated using ten independent repetitions.

- **Workload:** write-ratio and client-concurrency sweeps at three replicas. Write ratios are 0, 10, 50, 90, and 100 %, and concurrency ranges from 1 to 128.
- **Scaling:** replica-count sweep from three to nine replicas at 100 % writes and concurrency 32.
- **Conflict:** conflict-ratio sweep from 0 to 100 % at three replicas, 100 % writes, and concurrency 32.
- **Concurrency:** concurrency sweep from 1 to 256 at three replicas and 100 % writes.

The Workload and Concurrency families overlap in their concurrency range (1–128). They serve different purposes: the Workload family measures the interaction between write ratio and concurrency, so its concurrency range is bounded to keep the matrix tractable, while the Concurrency family isolates the concurrency dimension at 100 % writes and extends it to 256, where the throughput difference between the implementations is largest. The overlap also provides an additional check on run-to-run variability, since the same nominal configurations appear in both families.

- **Communication cost:** application-level one-way message-delay sweep over 0, 1, 3, 5, and 10 ms at three replicas, 100 % writes, and concurrency 32. A separate jitter sweep evaluates 0, 5, 10, 50, and 100 % jitter at the 5 ms delay level.
- **Election:** Raft-specific leader-election disruption. The current leader is terminated and transport to the surviving replicas is isolated for a configured period corresponding to a nominal target of 0–5 failed election attempts. The analysis uses the number of failed election attempts measured from the Raft adapter’s pre-vote and vote counters rather than treating the configured target as the realised number. Runs that do not recover within the measured phase are recorded as failed.
- **Failure:** controlled container termination at three replicas, 100 % writes, and concurrency 32. The scenarios comprise Raft leader failure, Raft follower failure, and EPaxos replica failure.

Failure injection is deterministic with respect to the experiment schedule. The runner terminates the selected container using `docker kill` at a configured offset within the measured phase. For Raft leader-failure experiments, the current leader is obtained from the Raft master and mapped to its corresponding container before termination. The client

subsequently continues issuing requests through the newly elected leader. For EPaxos replica-failure experiments, replica 0 is terminated and the client continues routing requests to the remaining replicas. Recovery intervals are reconstructed from request timestamps and the approximately 200 ms monitoring resolution and are therefore treated as approximate measurements.

3.7 Correctness Validation

The performance experiments evaluate throughput, latency, resource utilisation, and failure behaviour; they are not intended to provide a formal verification of consensus correctness. A separate correctness harness (`runner correctness`) is therefore used to check state convergence under the tested execution and failure scenarios.

The correctness harness reuses the Docker Compose infrastructure but runs short deterministic workloads outside the performance benchmark. It issues 2,000 requests, with each request writing a unique key derived from its request identifier and a deterministic value. Each replica exposes a read-only `GetState` RPC that returns a snapshot of its state machine and the number of applied commands. The harness compares the resulting replica states rather than internal logs, so the comparison does not assume that Raft and EPaxos expose equivalent log representations.

Each implementation is tested at concurrency levels 1, 8, and 32 and under the failure scenarios used by the failure experiment. Raft leader and follower replicas are killed and restarted during a run. EPaxos replicas are likewise killed and restarted. Because Raft persists its log, a restarted Raft replica is required to recover its previously committed state. EPaxos uses in-memory state under the evaluated configuration, so a restarted EPaxos replica is evaluated for liveness and subsequent state participation rather than persistence across restart.

This validation checks state convergence for the tested workloads and failure scenarios. It does not constitute a general linearizability proof or exhaustive verification of operation ordering.

3.8 Experimental Integrity and Statistical Analysis

Host-level telemetry, including load average and available memory, is recorded at approximately 200 ms resolution for every run. Host suspension, scheduling gaps, CPU starvation, and clock anomalies are detected and recorded in run metadata. Runs flagged as contaminated are not silently discarded.

The exclusion rule is fixed before aggregation and is applied uniformly across experimental conditions. A run whose host telemetry indicates experimental contamination is excluded from the corresponding aggregate, and a replacement run is executed so that the configuration retains its planned number of independent observations. The replacement is recorded as a separate attempt, including the reason for replacement, while the original observation remains in the

raw dataset. Excluded and replacement runs are reported in the accompanying execution-integrity record.

All aggregates are computed from raw per-run observations. Each configuration is evaluated using ten independent repetitions. Because a small number of repetitions stalled (Section 4), throughput and latency are summarised by the median of the per-repetition values, with 95 % confidence intervals obtained by non-parametric bootstrap (10,000 resamples). Protocol differences are tested with the exact Mann-Whitney U test, and workload-trend hypotheses with the Kruskal-Wallis test; no multiplicity correction is applied across the many configurations. Latency is reported using p50, p95, and p99 quantiles. Individual repetition values are retained and plotted where applicable.

The election experiment is treated as a Raft-specific recovery characterisation rather than as a direct protocol-family comparison. Its relationship between measured failed elections and availability gap is reported descriptively, with the individual observations shown alongside the fitted statistic. No causal interpretation is assigned to the measured association.

4 Evaluation

Unless otherwise stated, reported point estimates are medians over ten independent repetitions. Unless otherwise specified, results use three replicas, 100 % writes, and client concurrency 32. All figures are generated from the processed benchmark data by `lab/report/figures.py`. Figures show medians across repetitions, with individual repetition values overlaid where available.

4.1 Throughput and Scalability

Figure 1 shows measured throughput as the number of replicas increases. Throughput decreased with replica count for both evaluated implementations. At three replicas, the measured median was 5,292 req/s for EPaxos and 4,351 req/s for Raft. At five replicas, the medians were 5,443 and 3,150 req/s, respectively, while at nine replicas they were 2,094 and 2,113 req/s (Table 2). Relative to the three-replica measurement within this experiment family, the nine-replica median was 60 % lower for EPaxos and 51 % lower for Raft. The ratio of the two medians was 1.22 at three replicas, 1.73 at five, 1.18 at seven, and 0.99 at nine replicas. Thus, the measured throughput difference varied with replica count, and no statistically significant difference was detected at nine replicas.

We report medians rather than means because a small number of repetitions stalled. Across the entire dataset (1,302 runs), eight repetitions completed fewer than 10 % of the requests of their configuration’s median: two Raft runs (0.3 % of Raft runs) and six EPaxos runs (1.0 % of EPaxos runs). Seven of the eight were slow from start to finish with no failed requests, while one EPaxos run at nine replicas showed persistent request timeouts. The two Raft stalls coincided with elevated host load (load average 6.9–8.5 against 3–4 for healthy runs of the same configuration). The six EPaxos

stalls occurred at both low and high host load. None of the eight runs was flagged by the host-telemetry contamination rule, so they remain in the dataset. The means are therefore lower than the medians for both implementations, and the mean-based EPaxos/Raft ratio (1.29–1.75) is larger than the corresponding median ratio at seven and nine replicas. The observed stall counts (2 of 690 Raft runs and 6 of 612 EPaxos runs) were not significantly different (Fisher’s exact test, $p = 0.16$).

A second anomaly affects EPaxos at seven and nine replicas: request timeouts. Every EPaxos run at those sizes except one had 30–250 requests time out at the 2,000 ms client timeout, corresponding to success rates of 98.6–99.9 %, while Raft runs at the same sizes had no timeouts. Each timeout occupies a client worker for the full 2 s, reducing the concurrency available to successful requests. For a closed-loop client, Little’s law can be applied to the successful requests and timed-out requests separately. In particular,

$$N \approx X_{\text{ok}} W_{\text{ok}} + X_{\text{timeout}} W_{\text{timeout}},$$

where $X_{\text{timeout}} = N_{\text{timeout}}/20, \text{s}$ and $W_{\text{timeout}} = 2, \text{s}$. This accounting is consistent with every EPaxos run at seven and nine replicas. The successful requests themselves completed in 5.4–5.6 ms at p50, similar to the corresponding three-replica measurements. Thus, the reduced measured throughput is accounted for by requests occupying workers until the timeout rather than by an increase in the p50 latency of successful requests. The timeout pattern is observed only in the seven- and nine-replica EPaxos configurations; the available instrumentation does not establish the specific execution-path mechanism responsible for it.

The same nominal configuration (three replicas, 100 % writes, and concurrency 32) also occurs independently in several other steady-state experiment families. These repeated measurements provide an indication of run-to-run variability. The four family medians were 4,351, 4,368, 4,483, and 4,441 req/s for Raft, and 5,292, 5,288, 5,177, and 5,250 req/s for EPaxos. The EPaxos medians differed by approximately 2 % across these measurements, and the Raft medians by approximately 3 %. Cross-family variability was therefore of similar magnitude for the two implementations.

For the scaling family, the difference between the implementations was statistically significant at three and five replicas (exact Mann–Whitney U test, $p < 0.001$), while no statistically significant difference was detected at seven ($p = 0.12$) or nine ($p = 0.32$) replicas. The 95 % bootstrap confidence intervals of the medians overlap at seven and nine replicas (Table 2). The measured EPaxos/Raft throughput difference was therefore statistically significant only at the smaller cluster sizes in this experiment.

Figure 2 shows throughput across the client concurrency sweep. Throughput increased with concurrency for both implementations over the tested range. At concurrency 1, the measured medians were 304 req/s for Raft and 175 req/s for EPaxos. At concurrency 256, the medians were 19,333 req/s

Table 2: Throughput (req/s, median of 10 repetitions) by replica count, at 100 % writes and concurrency 32. Brackets show the 95 % bootstrap confidence interval of the median. Medians are used because several repetitions stalled (see Section 4); the mean is lower for both protocols at every replica count.

Replicas	Raft	EPaxos	EPaxos/Raft
3	4,351 [2,612–4,468]	5,292 [5,118–5,581]	1.22×
5	3,150 [2,771–3,346]	5,443 [4,469–5,611]	1.73×
7	2,590 [1,522–2,608]	3,053 [1,405–4,436]	1.18×
9	2,113 [1,450–2,160]	2,094 [1,386–3,074]	0.99×

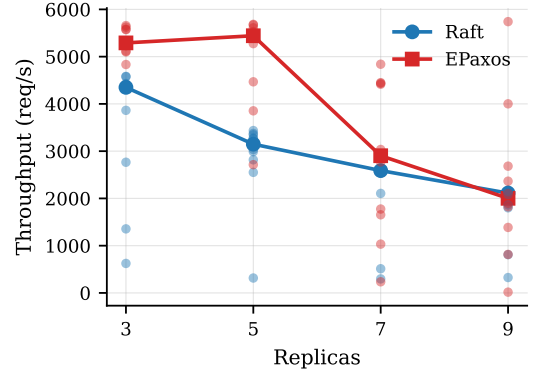


Figure 1: Throughput by replica count at 100 % writes and concurrency 32. Points show individual repetitions; the line marks the median.

and 41,077 req/s, respectively. The largest individual observations at concurrency 256 were 21,656 req/s for Raft and 43,681 req/s for EPaxos.

Between concurrency 64 and 256, the Raft median increased from 7,020 to 19,333 req/s, while the EPaxos median increased from 10,712 to 41,077 req/s. Both implementations therefore continued to gain throughput as concurrency increased, although the magnitude of the increase differed between the two implementations.

Figure 3 shows throughput across the write-ratio sweep. EPaxos medians ranged from 5,173 to 5,462 req/s, while Raft medians ranged from 4,220 to 4,463 req/s. Neither implementation exhibited a monotonic or otherwise consistent throughput trend across the tested write ratios. The highest EPaxos median occurred at 10 % writes, approximately 5 % above its 100 % write measurement, while the highest Raft median occurred at 0 % writes.

The variation across write ratios was not statistically significant. A Kruskal–Wallis test of the five write-ratio groups gave $H = 1.35$ for Raft and $H = 1.69$ for EPaxos, both below the χ^2_4 critical value of 9.49. The measurements therefore do not provide evidence of a systematic throughput effect associated with write ratio under the tested conditions.

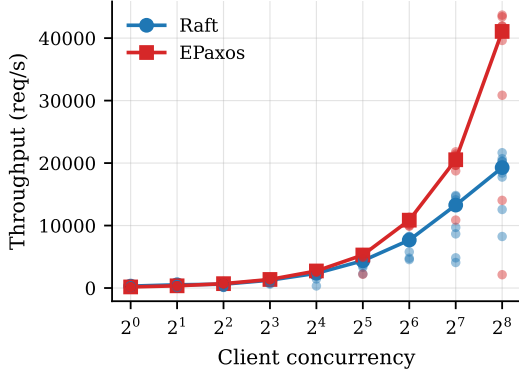


Figure 2: Median throughput by client concurrency at 3 replicas and 100 % writes. Points show individual repetitions.

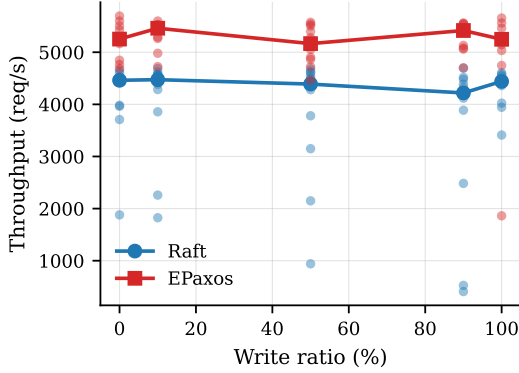


Figure 3: Throughput by write ratio at 3 replicas and concurrency 32. Points show individual repetitions; the line marks the median.

Figure 4 shows throughput across the configured conflict-ratio sweep. Median throughput ranged from 4,152 to 4,504 req/s for Raft and from 5,079 to 5,392 req/s for EPaxos. A Kruskal–Wallis test of the seven conflict-ratio groups gave $H = 3.71$ for Raft and $H = 3.18$ for EPaxos, both below the χ^2_6 critical value of 12.59. No monotonic throughput relationship with the configured conflict ratio was observed for either implementation.

4.2 Latency

Figure 5 shows median (p50) latency as the number of replicas increases. At three replicas, the median latency was 6.9 ms for Raft and 5.6 ms for EPaxos. Raft latency increased to 9.2 ms at five replicas and 12.0 ms at seven replicas, reaching 14.6 ms at nine replicas. EPaxos remained within a relatively narrow range of 5.4–5.6 ms across the tested replica counts (Table 3).

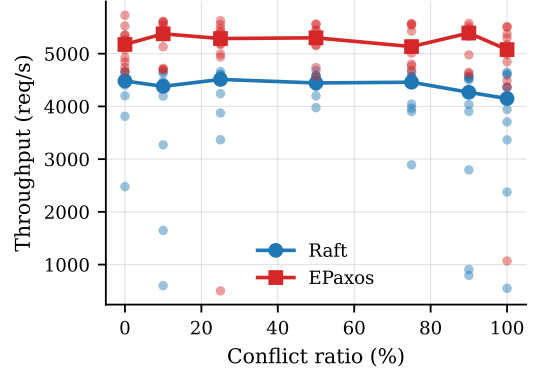


Figure 4: Throughput by conflict ratio at 3 replicas, 100 % writes, and concurrency 32. Points show individual repetitions; the line marks the median.

Table 3: Median (p50) latency in milliseconds by replica count, at 100 % writes and concurrency 32. Values are the median of the ten per-repetition p50 values.

Replicas	Raft p50 (ms)	EPaxos p50 (ms)
3	6.9	5.6
5	9.2	5.5
7	12.0	5.4
9	14.6	5.5

The evaluated Raft implementation therefore showed a larger change in median latency with replica count than the evaluated EPaxos implementation under this workload. The Raft p50 increased monotonically with replica count, whereas the EPaxos p50 remained within the 5.4–5.6 ms range.

The p50 values should be read together with the tail. The p99 was 21.4 ms for Raft and 23.8 ms for EPaxos at three replicas, and increased to 26–29 ms for Raft at seven to nine replicas while remaining at 14–16 ms for EPaxos. The median of the per-repetition p50 values reduces the influence of the stalled repetitions described earlier.

Figure 6 shows median latency across client concurrency. At concurrency 1, median latency was 2.8 ms for Raft and 5.5 ms for EPaxos. At concurrency 32, the corresponding values were 6.9 ms and 5.6 ms. At concurrency 256, they were 12.1 ms and 5.6 ms.

The relative latency ordering therefore changed with concurrency. Raft had the lower median at concurrency 1, while EPaxos had the lower median at concurrency 32 and above. At concurrency 256, the measured p50 was 5.6 ms for EPaxos and 12.1 ms for Raft.

As with throughput, the same nominal configuration (three replicas, concurrency 32) is measured by several experiment families. With medians, the p50 was consistent across families:

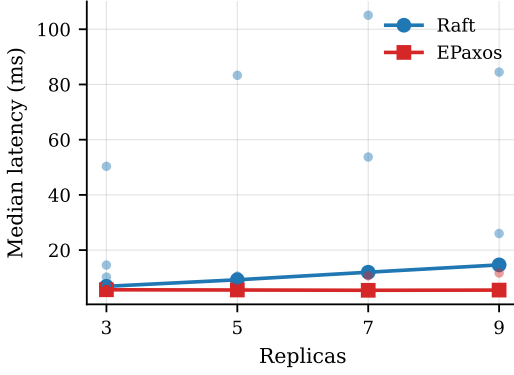


Figure 5: Median (p50) latency by replica count at 100 % writes and concurrency 32. Points show individual repetitions; the line marks the median.

Raft measured 6.9 ms in the scaling family and 6.9 ms in the concurrency family, while EPaxos measured 5.6 ms in both.

For a closed-loop client, Little’s law relates mean latency to concurrency and throughput as $W = N/X$. The reported p50 is the median rather than the mean, so it need not equal this quantity. At concurrency 256, the Raft p50 of 12.1 ms was close to N/X ($256/19,333 \approx 13.2$ ms), while the EPaxos p50 of 5.6 ms was below N/X ($256/41,077 \approx 6.2$ ms). This difference is compatible with a response-time distribution whose median is below its mean. We report p50, p95, and p99 in the processed dataset and use p50 here as the central-tendency summary.

At seven and nine replicas, the EPaxos p50 of 5.4–5.5 ms was substantially below N/X ($32/2,094\text{--}3,053 \approx 10.5\text{--}15.3$ ms). The difference is accounted for by the request timeouts described above. Every EPaxos run at these sizes except one had requests time out at the 2,000 ms client timeout, and each timeout occupies a worker for the full 2 s. For a closed-loop client with concurrency N , the corresponding accounting is

$$N \approx X_{\text{ok}}W_{\text{ok}} + X_{\text{timeout}}W_{\text{timeout}},$$

where $X_{\text{timeout}} = N_{\text{timeout}}/20, \text{s}$ and $W_{\text{timeout}} = 2, \text{s}$. This accounting is consistent with every EPaxos run at seven and nine replicas. The low throughput can therefore be accounted for without requiring an increase in the p50 latency of successful requests. Raft runs at the same sizes had no timeouts, and their measured N/X values were closer to their reported p50 latencies.

4.3 Resource Utilisation

Figure 7 summarises the per-replica resource measurements. Across the evaluated runs, mean per-replica CPU utilisation was 35.8 % for Raft and 35.0 % for EPaxos. Mean peak per-replica RSS was 26.4 MB for Raft and 46.8 MB for EPaxos. Mean per-replica network rates were 441 kB/s RX and 443 kB/s TX for Raft, compared with 408 kB/s RX and 392 kB/s TX for EPaxos.

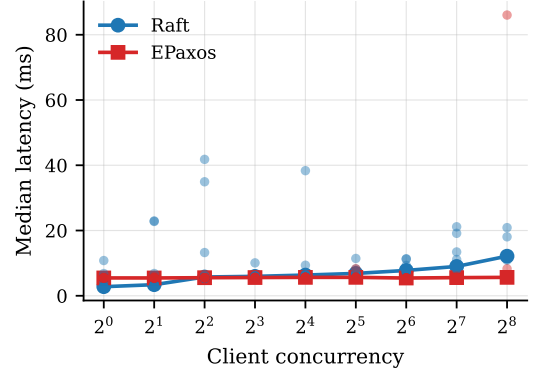


Figure 6: Median (p50) latency by client concurrency at 3 replicas and 100 % writes. Points show individual repetitions; the line marks the median.

Figure 8 examines the distribution of CPU utilisation in the nine-replica scaling experiment. Raft per-replica means ranged from 16.3 % to 29.4 %. The replica serving as leader during a given repetition generally consumed less CPU than the followers. Labelling each run by the leader reported at the start of its measured phase, the leader/follower CPU ratio was 0.61–0.64 at three replicas, 0.70 at five replicas, and 0.73–0.75 at seven and nine replicas, using the medians over the ten repetitions.

This pattern is consistent with the request path of the evaluated HashiCorp Raft implementation, in which the leader sends AppendEntries and waits for quorum responses while followers receive, persist, and apply replicated log entries. The measurements do not isolate the contributions of persistence, state-machine application, or communication processing to the observed CPU difference. They therefore establish that the leader did not form the CPU hotspot in the evaluated implementation, but do not by themselves identify the individual processing components responsible for the difference.

4.4 Failure Recovery Behaviour

Failure outcomes differed by failure type (Table 4, Figure 9). Killing the Raft leader produced a write-availability gap of 1.81–4.74 s and 2,169–4,133 failed requests. Killing a Raft follower produced a gap of 0.02–0.04 s with no failed requests. Killing an EPaxos replica produced a gap of 0.02–0.09 s with 10–106 failed requests.

All runs completed successfully according to the experiment harness. The reported failed requests correspond to requests issued during the transition and are therefore distinct from run-level failure.

The longer availability gap following Raft leader termination coincided with the configured Raft election and request-retry behaviour. In contrast, the tested follower-failure case did not require replacement of the Raft leader, while the EPaxos replica-failure case continued with the remaining replicas. These results describe the tested failure scenarios

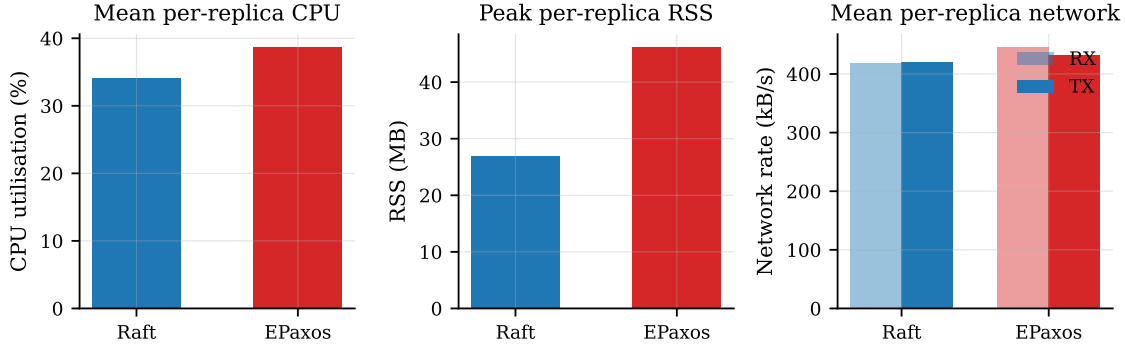


Figure 7: Mean per-replica CPU utilisation, peak per-replica RSS, and mean per-replica network rates, aggregated over the evaluated runs.

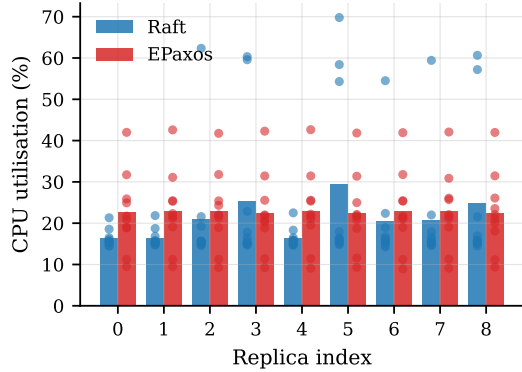


Figure 8: Per-replica CPU utilisation in the 9-replica scaling runs. Bars show means over repetitions; points show individual repetitions.

Table 4: Failure behaviour at 3 replicas, 100 % writes, concurrency 32. Gap is the largest interval with no successful write completion after failure injection.

Failure	Protocol	Gap (s)	Failed req.
Leader kill	Raft	1.81–4.74	2,169–4,133
Follower kill	Raft	0.02–0.04	0
Replica kill	EPaxos	0.02–0.09	10–106

and should not be interpreted as representative of all leader-based or leaderless implementations.

The election experiment provides a more controlled characterisation of Raft-specific recovery under repeated election disruption. The measured number of failed elections ranged from 0 to 13 across the 60 runs. The availability gap was strongly associated with the measured election count ($r = 0.90$, $p < 10^{-4}$), but this relationship is partly mechanical because the isolation interval itself increases with the configured disruption duration.

To examine recovery independently of the imposed isolation interval, the analysis also measures the time from the end of the isolation window to the first successful request. This post-isolation recovery interval was more variable and showed a weaker positive association with the measured failed election count ($r = 0.41$, $p = 0.001$, $n = 60$). This result is reported as an observed association rather than as evidence of a causal relationship.

4.5 Correctness Validation

The performance experiments are not intended to constitute a formal verification of consensus correctness. The separate correctness harness described in Section 3 instead checks state convergence under the tested workloads and failure scenarios.

The harness issued 2,000 deterministic requests per test. Each request used a unique request-derived key and deterministic value, allowing the expected state to be reconstructed independently. After each test, the harness collected the state-machine snapshot from each available replica through the read-only administration RPC and compared the resulting states without relying on internal log representations.

The harness tested concurrency levels 1, 8, and 32 and the failure scenarios used by the failure experiments. Raft leader and follower replicas were killed and restarted, while an EPaxos replica was likewise killed and restarted. For Raft, the restarted replica was required to recover the committed state from its persistent log. For EPaxos, the evaluated in-memory configuration was required to resume participation after restart; persistence across restart was not part of this validation.

All 18 correctness tests passed (Table 5). The tests therefore provide evidence that the evaluated implementations converged to the expected state for the tested request and failure scenarios and resumed processing after the injected faults.

This validation does not establish general linearizability, exhaustive ordering correctness, or correctness under workloads outside the tested cases. In particular, because the

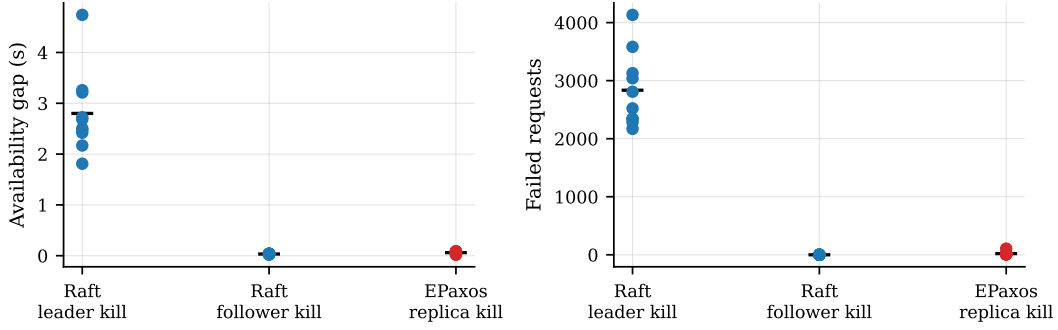


Figure 9: Write-availability gap and failed requests by failure type at 3 replicas, 100 % writes, and concurrency 32. Points show individual repetitions; the bar marks the mean.

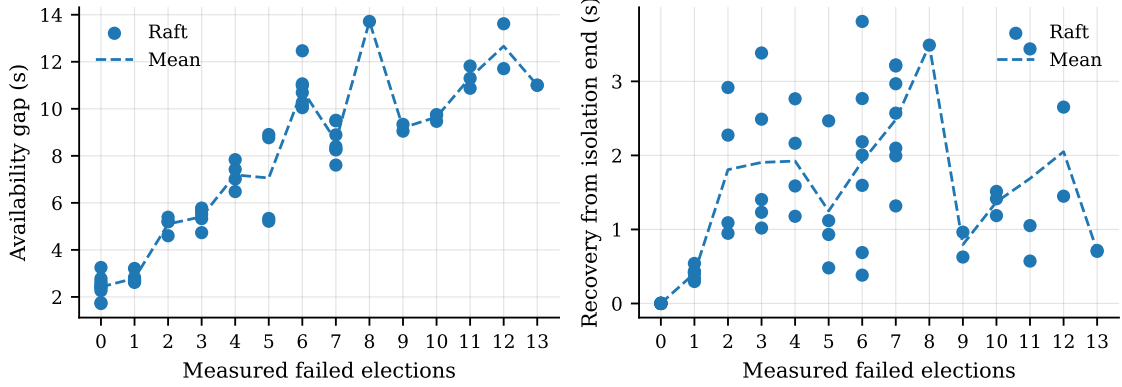


Figure 10: Election-failure recovery in Raft at 3 replicas, 100 % writes, and concurrency 32. Left: write-availability gap versus the measured number of failed elections. Right: recovery time from the end of the isolation window to the first successful request. Points show individual repetitions; the dashed line marks the mean per value.

Table 5: Correctness-validation results: 2,000 deterministic requests per test at 3 replicas, 100 % writes. Every replied request was applied exactly once on every replica, surviving replicas agreed on the final state, and requests issued after each fault were processed normally.

Implementation	Sequential	c=8	c=32	Fault
Raft (HashiCorp)	PASS	PASS	PASS	follower/leader
Raft (etcd)	PASS	PASS	PASS	follower/leader
EPaxos (original)	PASS	PASS	PASS	replica
EPaxos (nvb)	PASS	PASS	PASS	replica

correctness workload assigns a unique key to each request, it primarily tests state convergence rather than providing an independent verification of operation-order semantics.

4.6 Communication Cost

The communication-cost family evaluates sensitivity to additional application-level one-way message delay. It should therefore be interpreted as a controlled communication-condition experiment rather than as a direct simulation of a physical network.

Figure 11 shows mean throughput at each delay level with zero jitter for four implementations. The implementations exhibited substantially different sensitivity to the injected delay. HashiCorp Raft decreased from 4,502 req/s at zero delay to 1,280 req/s at 10 ms, a 72 % reduction. nvb EPaxos decreased from 4,346 to 833 req/s, an 81 % reduction, while the original EPaxos implementation decreased from 5,527 to 221 req/s, a 96 % reduction. The etcd Raft implementation showed a different response: its throughput decreased from 6,420 req/s at zero delay to 375 req/s at 1 ms and 45 req/s at 10 ms.

The measured ordering therefore differed across implementations under added communication delay. The 95 %

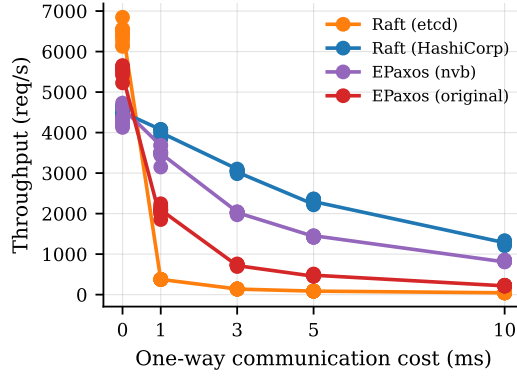


Figure 11: Mean throughput by application-level one-way communication delay at 3 replicas, 100 % writes, and concurrency 32 for the four evaluated implementations.

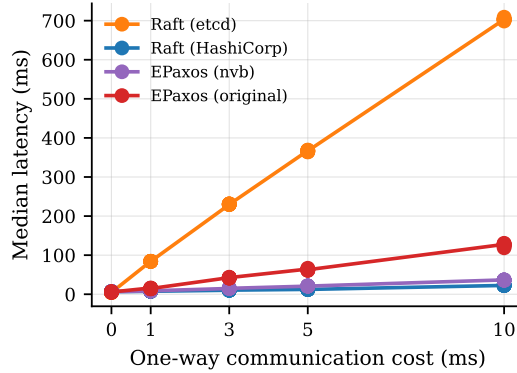


Figure 12: Median (p50) latency by application-level one-way communication delay at 3 replicas, 100 % writes, and concurrency 32 for the four evaluated implementations.

confidence intervals of the means were separated between the evaluated implementations at every non-zero delay level.

Median latency increased with the injected delay for all four implementations. At 10 ms, the measured p50 values were 22.6 ms for HashiCorp Raft, 36.2 ms for nvb EPaxos, 125.9 ms for original EPaxos, and 703.4 ms for etcd Raft (Figure 12). The implementation ordering observed for latency at this condition corresponded to the throughput ordering.

The jitter sweep produced a smaller change than the corresponding increase in fixed delay. At a 5 ms base delay, increasing jitter from 0 to 100% reduced throughput from 2,316 to 1,760 req/s for HashiCorp Raft (24%), from 1,432 to 1,082 req/s for nvb EPaxos (24%), from 479 to 305 req/s for original EPaxos (36%), and from 87 to 59 req/s for etcd Raft (32%).

Under the implemented jitter model, 100 % jitter increases the maximum additional delay from zero to 5 ms and increases

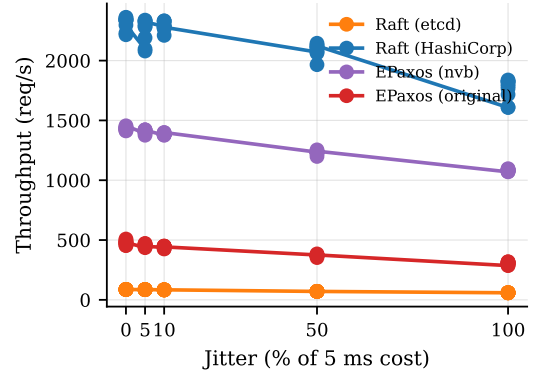


Figure 13: Mean throughput by jitter at a 5 ms application-level one-way delay, 3 replicas, 100 % writes, and concurrency 32 for the four evaluated implementations.

the mean injected delay from 5 to 7.5 ms. The measured throughput changes are therefore consistent with an increase in average injected delay. The experiment does not separate the effect of delay variance from the corresponding change in mean delay.

4.7 Summary of Findings

The measurements support the following observations under the evaluated conditions:

- Throughput decreased as replica count increased for both evaluated implementations. In the scaling experiment, the nine-replica median was 60 % below the three-replica median for EPaxos and 51 % below it for Raft.
- The relative throughput difference between the evaluated Raft and EPaxos implementations varied with replica count. The EPaxos/Raft median ratio was 1.22 at three replicas, 1.73 at five, 1.18 at seven, and 0.99 at nine, where no statistically significant difference was detected.
- Eight repetitions across the dataset stalled (two Raft and six EPaxos), motivating the use of medians for the reported point estimates. The Raft stalls coincided with elevated host load, whereas the EPaxos stalls occurred at both low and high host load. The mean-based EPaxos/Raft ratio was 1.29–1.75 and was larger than the corresponding median ratio at seven and nine replicas.
- Throughput continued to increase with client concurrency up to the tested maximum of 256. At that point, the measured medians were 41,077 req/s for EPaxos and 19,333 req/s for Raft.
- Median latency depended on both concurrency and replica count. EPaxos had the lower measured median at the baseline configuration and throughout the replica-scaling measurements. At concurrency 256,

the p50 was 5.6 ms for EPaxos compared with 12.1 ms for Raft.

- Write ratio and conflict ratio did not produce a systematic throughput trend over the tested ranges. Kruskal–Wallis tests of the write-ratio and conflict-ratio groups were not significant for either implementation ($H \leq 3.71$ against critical values of 9.49 and 12.59).
- Raft leader termination produced a longer availability gap than the tested Raft follower and EPaxos replica termination scenarios. The additional election experiment showed an observed association between failed-election count and recovery intervals, although the relationship was not interpreted causally.
- Per-replica CPU utilisation was distributed differently in the nine-replica scaling experiment. Across the scaling configurations, the measured leader/follower CPU ratio remained below 1.0 (0.61–0.64 at three replicas, 0.70 at five, and 0.73–0.75 at seven and nine). Thus, the leader did not form the CPU hotspot in the evaluated HashiCorp Raft implementation. These measurements do not by themselves identify CPU saturation as a throughput bottleneck.
- Communication sensitivity differed substantially across the four evaluated implementations. At 10 ms of application-level one-way delay, throughput reductions ranged from 72 % for HashiCorp Raft to 96 % for the original EPaxos implementation, while etcd Raft showed a much steeper decline beginning at the smallest non-zero delay tested.
- The communication-cost results show that implementation choice can materially change the measured performance relationship between leader-based and leaderless consensus. Results from a single implementation of each protocol should therefore be interpreted as measurements of those implementations rather than as implementation-independent properties of the corresponding protocol families.

5 Discussion

5.1 RQ1: Workload and Scaling Sensitivity

5.1.1 Read/Write Composition. The read/write-ratio experiment (Figure 3) did not reveal a systematic monotonic relationship between write ratio and throughput for either implementation. Across the tested range, EPaxos throughput varied between 5,173 and 5,462 req/s, while Raft varied between 4,220 and 4,463 req/s. A Kruskal–Wallis test of the five write-ratio groups is not significant for either implementation ($H = 1.35$ for Raft, $H = 1.69$ for EPaxos, against a χ^2_4 critical value of 9.49). Consequently, the experiment does not provide strong evidence that changing the read/write composition alone produces a consistent throughput trend.

An exploratory second-order regression captured some curvature in the measured responses more effectively than a first-order model. However, the fitted curvature should be interpreted descriptively rather than as evidence of a

protocol-level effect. The observed variation is relatively small compared with the cross-family variability, and the experiment does not directly measure the internal processing costs associated with reads and writes.

The absence of a strong workload-composition effect is also consistent with the execution model used in this study. Reads are routed through consensus in both implementations rather than being served through a Raft-specific local-read optimisation. Consequently, the read operations exercised the same consensus-facing request path rather than providing a separate local-read fast path. This design improves comparability between the two implementations, but it also limits the extent to which the experiment can isolate read-specific consensus costs.

The write-dominated endpoint is nevertheless useful for interpreting the other experiments. In the evaluated Raft configuration, requests are directed to the elected leader, whereas EPaxos permits proposals to be initiated at multiple replicas. This difference becomes more directly observable in the concurrency and replica-scaling experiments than in the read/write sweep itself.

Overall, the read/write experiment indicates that workload composition alone is not sufficient to explain the observed performance differences. The measurements instead suggest that concurrency, replica count, and implementation-specific processing behaviour can be at least as important as the aggregate read/write ratio under the tested conditions.

5.1.2 Replica Scaling. Increasing the replica count reduced throughput for both protocols (Figure 1, Table 2). At the 3-replica configuration, EPaxos measured 5,292 req/s compared with 4,351 req/s for Raft. At 9 replicas, the corresponding medians were 2,094 and 2,113 req/s. Thus, increasing the replica count reduced the measured throughput of both implementations rather than providing additional aggregate processing capacity.

The decrease is expected because larger replica sets require additional communication and coordination. The experiment therefore does not support interpreting replica addition as a direct source of throughput scaling for either approach. The relative difference between the implementations also became smaller at the largest tested replica counts: the EPaxos/Raft median ratio decreased from 1.22 at three replicas to 0.99 at nine replicas, and the difference was not statistically significant at seven or nine replicas (exact Mann–Whitney $p = 0.12$ and $p = 0.32$). The EPaxos advantage is therefore established only at three and five replicas.

The Raft configuration has a distinct request-routing structure. Client requests are directed to the elected leader, so increasing the number of followers does not create additional independent request-processing endpoints. Additional replicas increase redundancy and replication capacity, but the normal request path remains centred on one leader.

EPaxos distributes proposal initiation across replicas, but this does not imply that its throughput should remain constant as the cluster grows. Larger replica sets also increase

the coordination and communication associated with each operation. In addition, the evaluated implementation required a dependency-set extension for seven- and nine-replica configurations. The measurements at those sizes therefore include both the effects of larger clusters and the implementation required to support those configurations. The EPaxos throughput decline at seven and nine replicas is specifically associated with request timeouts: every EPaxos run at those sizes except one had 30–250 requests time out at the 2,000 ms client timeout, and each timeout occupies a client worker for the full 2s, reducing the concurrency available to successful requests (Section 4). The successful requests themselves complete in 5.4–5.6 ms, the same as at three replicas, so the decline is not a uniform slowdown of the EPaxos execution path. We did not instrument the EPaxos execution path sufficiently to establish whether the timeouts are caused by the dependency-set extension or by another mechanism.

The scaling experiment consequently shows a common cost of increasing replica count while also exposing a structural difference in request placement. It does not, however, establish a general scaling advantage for either consensus family.

5.1.3 Client Concurrency. Client concurrency produced a larger separation between the evaluated implementations than the read/write-ratio experiment (Figure 2). At concurrency 1, throughput was 304 req/s for Raft and 175 req/s for EPaxos. At concurrency 256, the corresponding medians were 19,333 and 41,077 req/s.

The increase in throughput with concurrency indicates that both implementations had additional processing capacity available at low concurrency. The larger increase observed for EPaxos at high concurrency is consistent with its ability to accept proposals at multiple replicas, whereas the evaluated Raft configuration directs all benchmark requests through the elected leader.

This result should nevertheless be interpreted as an implementation-level observation. The benchmark measures a complete request-processing path, including serialisation, transport, state-machine processing, consensus processing, and scheduling. The experiment does not independently measure which component becomes the limiting resource as concurrency increases.

The latency measurements provide a complementary view. Median latency increased for Raft at high concurrency, reaching 12.1 ms at concurrency 256, while the EPaxos p50 stayed at 5.6 ms. Thus, the additional throughput obtained from higher concurrency was accompanied by increased request latency for Raft but not for EPaxos.

Taken together, the concurrency results indicate that the difference between centralised and distributed request placement becomes more visible under high offered concurrency in the evaluated implementations. They do not establish that leaderless consensus universally scales better with concurrency, but they identify concurrency as a condition under which the request-placement structure materially affects the measured throughput.

5.1.4 Conflict Rate. The conflict-ratio experiment did not reveal a clear monotonic relationship between conflict rate

and throughput (Figure 4). Raft medians ranged from 4,152 to 4,504 req/s, while EPaxos medians ranged from 5,079 to 5,392 req/s. A Kruskal-Wallis test of the seven conflict-ratio groups is not significant for either implementation ($H = 3.71$ for Raft, $H = 3.18$ for EPaxos, against a χ^2_6 critical value of 12.59).

These measurements therefore do not support a simple claim that increasing conflict consistently reduces throughput. In particular, the observed values fluctuate across the tested conflict levels rather than following a smooth response curve.

The result is notable for EPaxos because conflict detection and dependency management are central components of its execution model. However, the present benchmark does not directly record dependency-set sizes, fast-path versus slow-path decisions, or the number of commands requiring additional coordination. Consequently, changes in throughput cannot be attributed to those mechanisms without further instrumentation.

The conflict experiment is therefore best interpreted as evidence that no strong monotonic conflict-throughput relationship was observed under the tested conditions. More detailed conclusions about how conflict affects EPaxos would require measurements of the internal execution paths associated with conflicting and non-conflicting operations.

5.2 RQ2: Failure Behaviour

The failure experiments show a clear distinction between leader failure and non-leader replica failure in the evaluated configurations (Table 4, Figure 9). Killing the Raft leader produced an availability gap of 1.81–4.74 s and 2,169–4,133 failed requests. In contrast, killing a Raft follower produced gaps of 0.02–0.04 s with no failed requests, while EPaxos replica failures produced gaps of 0.02–0.09 s with 10–106 failed requests.

The Raft leader-failure result follows from the request path used by the benchmark. Normal requests are routed to the elected leader, so terminating that replica removes the endpoint currently responsible for processing requests. Service therefore depends on failure detection, leader election, and subsequent client reconnection. The measured gap consequently reflects both consensus recovery and the surrounding failure-handling mechanisms.

The follower-failure experiment provides a useful contrast. With three Raft replicas, removal of a follower leaves the elected leader available and leaves the cluster with the quorum required by the tested configuration. The normal request path can therefore continue without requiring a leader transition.

The EPaxos result differs in request placement because no single replica is required to serve as the normal proposal endpoint. Removing one replica therefore does not require election of a replacement leader before other replicas can continue processing requests. The short interruption observed in the experiment is consistent with this structural difference, although the measured failed-request count also depends on client timing and retry behaviour.

The election experiment further characterises the Raft-specific recovery path. The measured number of failed election attempts was strongly correlated with the total availability gap ($r = 0.90$, $n = 60$). This relationship is partly mechanical because the failure injection itself increases the isolation duration for larger target values. A second analysis measured recovery from the end of the isolation period to the first successful request. This residual recovery time showed a weaker but still statistically detectable association with the measured number of failed elections ($r = 0.41$, $p = 0.001$).

The latter result should be interpreted cautiously. The measured election count is itself an outcome of the recovery process rather than an independently manipulated workload variable. The correlation therefore characterises the observed Raft recovery behaviour but should not be interpreted as a causal estimate of the effect of an additional failed election.

Overall, the failure experiments identify a specific availability dependency in the evaluated Raft architecture: failure of the normal request-processing replica has a substantially larger effect than failure of a follower. EPaxos avoids this particular single-endpoint dependency, but replica failures still introduce interruption and failed requests. The results therefore distinguish failure modes rather than establishing that one consensus family has universally lower failure-recovery cost.

5.3 RQ3: Implementation Sensitivity

The implementation-comparison experiment demonstrates that measured performance can differ substantially between implementations of the same consensus family. This effect is particularly visible in the communication-cost experiment (Figure 11), where two Raft implementations and two EPaxos implementations responded differently to the same injected communication conditions.

At zero injected cost, the measured throughputs were 6,420 req/s for etcd Raft, 5,527 req/s for the original EPaxos implementation, 4,502 req/s for HashiCorp Raft, and 4,346 req/s for nvb EPaxos. Thus, the ordering at the local-network baseline was not determined solely by whether an implementation was leader-based or leaderless.

The divergence became larger as communication cost increased. At 10 ms of injected one-way cost, HashiCorp Raft measured 1,280 req/s, nvb EPaxos 833 req/s, and the original EPaxos implementation 221 req/s. The etcd Raft implementation measured only 45 req/s at the same cost, after already falling from 6,420 req/s at zero cost to 375 req/s at 1 ms.

These results are important because implementations within the same protocol family exhibit substantially different responses to the same experimental condition. The observed differences therefore cannot be attributed solely to the abstract distinction between leader-based and leaderless consensus. They arise from the complete implementation stack, including request processing, message handling, persistence, scheduling, transport behaviour, and other implementation choices.

The communication experiment also shows that absolute performance and sensitivity to perturbation are separate properties. An implementation with higher throughput under local conditions does not necessarily exhibit the smallest degradation when communication delay is introduced. Consequently, a single baseline benchmark can conceal substantial differences in behaviour under changed operating conditions.

Jitter produced a smaller effect than the fixed communication cost at the tested 5 ms operating point. Increasing jitter from 0% to 100% reduced throughput by approximately 24–36% across the four implementations. Because the implemented jitter increases the mean delay from 5 ms to 7.5 ms at 100% jitter, part of this reduction can be attributed to the increased mean communication delay. The experiment therefore does not isolate variance in delay from the accompanying change in mean delay.

The implementation results consequently place an important limitation on protocol-level comparisons. The measurements demonstrate how the evaluated implementations behave under controlled conditions; they do not permit the absolute throughput of one implementation to be interpreted as an intrinsic throughput bound of its underlying consensus algorithm.

5.4 Overall Interpretation

Taken together, the three research questions show that the measured difference between leader-based and leaderless consensus is multidimensional and strongly dependent on experimental conditions.

The workload and scaling experiments show that increasing replica count reduced throughput for both evaluated implementations, while increasing client concurrency exposed a substantially larger throughput difference at high concurrency. The Raft request path remains centred on a single elected leader, whereas EPaxos permits proposal initiation at multiple replicas. This structural difference provides a plausible explanation for why request placement becomes more visible under high concurrency, but the measurements do not isolate this mechanism from other implementation-level costs.

The failure experiments identify a second consequence of centralised request processing. In the evaluated Raft configuration, losing the leader removes the endpoint used for normal request processing and requires leader recovery before normal service resumes. Losing a follower does not have the same effect. EPaxos does not have an equivalent single leader endpoint, and the tested replica failure consequently produced a much shorter interruption. This difference concerns a specific failure mode rather than overall fault-tolerance superiority.

The implementation comparison provides the strongest qualification to any direct protocol ranking. The four implementations showed substantially different throughput and latency responses to communication delay, and the zero-delay baseline itself did not preserve a simple leader-based versus leaderless ordering. Implementation choices therefore materially influence the observed results.

The evidence is consequently better interpreted as a measurement of *implementation behaviour under controlled conditions* than as a universal ranking of Raft and EPaxos. The experiments show that centralised leadership creates a specific request-routing and failure dependency in the evaluated Raft configuration, while distributed proposal placement changes how request concurrency and replica failure are exposed to the system. At the same time, persistence, transport, scheduling, and other implementation mechanisms can amplify, reduce, or even reverse observed performance differences.

Accordingly, the central finding of this study is not that either leader-based or leaderless consensus is intrinsically faster. Instead, the measurements demonstrate that the observed performance relationship depends jointly on consensus structure, implementation, workload, replica count, concurrency, communication conditions, and failure mode. A meaningful comparison therefore requires evaluating both absolute performance and the response of each implementation to controlled changes in its operating conditions.

6 Threats to Validity

6.1 External Validity

All replicas run on a single host over a bridge network, so the measurements contain no wide-area latency, no real packet loss, and no multi-machine failure domains. The host is a WSL2 environment with 12 physical cores, which places a virtualisation layer between the containers and the underlying hardware. Cluster sizes are 3–9, the keyspace is 1,000 keys, commands are uniform in size, and the measured phase is 20s. The results therefore describe short, uniform, single-host runs rather than long-lived deployments with skewed access patterns, large payloads, heterogeneous hardware, or geographically distributed replicas.

The host has 12 cores, and the configured CPU quotas exceed this at the larger cluster sizes: nine replicas at 2.0 CPUs each, plus the client and the master, request 21 CPUs in total. The quotas are not evidence of saturation, however: the measured host load average stayed at 3.8–5.1 across the scaling family (against 12 cores), and the per-replica CPU measurements in Section 4 (means of 16–29% for Raft and 22–23% for EPaxos at nine replicas) imply roughly $9 \times 0.25 \times 2 \approx 4\text{--}5$ cores of actual demand. The host was therefore not CPU-saturated, and the throughput decline with replica count is not explained by host saturation. The quotas do mean that a single replica can burst to 2.0 CPUs, so per-replica CPU readings above 100% of one core are possible; the reported per-replica means are well below that.

The benchmark also places multiple replicas, the client, and the control processes on the same host. Consequently, processes may compete for host CPU resources, and measured resource utilisation can depend on host-level scheduling and contention. The experiments detect anomalous resource conditions, but do not provide strict CPU or hardware isolation. Resource utilisation should therefore be interpreted as an observation under the evaluated environment rather than as an implementation-independent resource requirement.

Each replica is allocated two CPUs, while the evaluated processes use `GOMAXPROCS=4`. Thus, the configured Go scheduler parallelism exceeds the nominal container CPU quota. The resulting interaction with host scheduling and cgroup CPU throttling is part of the measured environment and may affect both throughput and latency. Repeating the experiments with `GOMAXPROCS` matched to the CPU quota would provide an additional sensitivity check.

Reads are routed through consensus in both protocols and no Raft-specific read optimisation is enabled. The read comparison is therefore one of equivalent consensus-facing request paths rather than of each system’s fastest possible read path. A deployment using mechanisms such as leader leases or `ReadIndex` could exhibit a different read profile. Similarly, both systems are measured with identical CPU and memory allocations. This provides a controlled comparison but does not represent a resource-tuned deployment of either implementation.

The comparison does not establish that the measured behaviour is representative of all Raft or EPaxos implementations. The implementation-sensitivity experiments explicitly demonstrate that batching, persistence, networking, scheduling, transport handling, configuration defaults, and other implementation choices can substantially affect measured performance. Consequently, absolute throughput and latency should be interpreted as properties of the evaluated implementations and environment rather than as intrinsic properties of the underlying protocols.

Persistence is a particular source of asymmetry between the two protocols. Both Raft implementations persist their log (HashiCorp Raft to BoltDB, etcd Raft to a write-ahead log with batched fsync), while both EPaxos implementations keep state in memory. The Raft adapters also compact their logs: HashiCorp Raft snapshots at 8,192 entries and keeps 1,024 trailing logs, and the etcd adapter compacts below the same trailing threshold. At the measured throughputs (roughly 2,000–4,000 req/s), a snapshot is triggered every 2–4s, so snapshot and compaction work is part of the steady-state Raft path. The latency and throughput differences between Raft and EPaxos therefore include the cost of persistence and log maintenance, which is a property of the evaluated implementations, not of the consensus protocols themselves.

The EPaxos implementation is additionally modified to support the dependency-set sizes required by the 7- and 9-replica configurations. These modifications are part of the evaluated implementation and may contribute to the observed performance at those replica counts. Results at larger cluster sizes should therefore not be interpreted as isolating the cost of the EPaxos protocol from the cost of this implementation extension.

The communication experiment introduces delay in the application-level outbound message path rather than through a physical network emulator. The injected delay therefore does not reproduce all effects of real networks, including queueing, packet loss, link-level scheduling, or interactions among independent flows. In addition, the jitter experiment changes the mean delay when jitter is increased. Its results

consequently cannot isolate the effect of delay variance from the accompanying change in mean delay. A network-level experiment using an emulator such as `tc netem`, or an equivalent asynchronous delay mechanism, would provide stronger external validity.

The failure and leader-election experiments are conducted with three replicas. Their recovery times therefore characterise the evaluated three-replica configuration and should not be interpreted as establishing recovery behaviour for larger clusters. The failure scenarios are also not symmetric: the Raft leader-failure experiment removes the replica used as the normal request endpoint, whereas the Raft follower-failure and EPaxos replica-failure experiments remove a non-central replica. The results therefore characterise specific failure modes rather than providing a complete cross-protocol fault-tolerance comparison.

For Raft, the observed leader-recovery interval also depends on the configured heartbeat and election-timeout parameters, as well as client timeout and reconnection behaviour. The reported recovery times should therefore be interpreted relative to the evaluated timing and request-handling configuration rather than as protocol-independent bounds on Raft recovery.

Finally, the correctness harness uses deterministic unique-key operations and compares replica state after execution. It verifies state convergence under the tested workloads and restart scenarios, but it does not constitute a formal verification of consensus correctness or establish general linearizability. In particular, the unique-key workload does not exercise conflicting operations on the same state in a way that would expose every possible ordering violation.

6.2 Statistical Conclusion Validity

Each configuration is measured ten times, and aggregates are reported as medians with non-parametric bootstrap 95% confidence intervals. Ten repetitions leave substantial uncertainty for a variable system: measuring the same nominal configuration in four steady-state experiment families produced medians that differed by 2–3% for both protocols, while the bootstrap intervals overlap in several comparisons whose medians differ, such as at seven and nine replicas. We therefore interpret differences in the context of observed run-to-run variability and treat overlapping comparisons as inconclusive rather than as evidence of a small but established effect.

The experiment suite contains many configurations and pairwise comparisons without a global multiplicity correction. An isolated statistically detectable difference should therefore not be interpreted as strong evidence for a general effect. This consideration is particularly important for exploratory analyses such as regression over workload composition and correlation analyses within the election experiment.

The election correlation analysis also requires particular caution. The measured number of failed election attempts is an outcome of the recovery process and is mechanically related to the injected isolation duration. Accordingly, the reported correlations characterise associations within the observed Raft recovery runs rather than providing causal

estimates of the effect of an additional failed election. In addition, the 60 election runs are grouped by the configured isolation target (six targets, ten runs each), so the observations are not fully independent: runs sharing a target share the injection duration, which inflates the effective sample size and makes the reported p -values optimistic. The correlation should therefore be read as a descriptive association, not as a hypothesis test with 60 independent observations.

The 20 s measurement window also limits the extent to which the experiments characterise long-term steady-state behaviour. The reported results should therefore be understood as measurements over the specified workload and measurement interval rather than as estimates of indefinite long-running performance.

The benchmark uses a closed-loop client in which each worker issues its next request only after the previous request completes. Configured concurrency therefore represents a maximum number of in-flight requests rather than an independently controlled arrival rate. Throughput and latency results should accordingly not be interpreted as a conventional open-loop capacity curve.

Outliers within clean runs are not removed. Runs identified as affected by environmental interference or measurement failure are investigated separately and are not excluded solely because their performance is inconvenient for the comparison. Replacement runs are recorded separately, and the original observations remain in the raw dataset. All reported aggregates are derived from the retained per-run observations.

7 Conclusion

This paper presented an experimental comparison of Raft and EPaxos as representative leader-based and leaderless consensus approaches. Rather than treating a single throughput measurement as representative of a consensus protocol, the evaluation examined throughput, latency, replica scaling, client concurrency, failure recovery, resource utilisation, and communication sensitivity under controlled variations in workload and system conditions.

The workload and scaling experiments show that the measured performance relationship depends on the operating condition. Increasing the replica count reduced throughput for both evaluated implementations, while increasing client concurrency produced a substantially larger throughput increase for EPaxos at high concurrency. The evaluated Raft configuration routes normal requests through an elected leader, whereas EPaxos permits proposal initiation at multiple replicas. This structural difference is consistent with the larger throughput response observed for EPaxos under high concurrency, but the measurements do not establish that distributed proposal processing is the sole cause of the difference. The read/write and conflict experiments further show that workload composition does not necessarily produce a simple or monotonic throughput response.

The failure experiments identify a distinct operational consequence of the leader-centred request path. Failure of the Raft leader produced an availability gap of 1.81–4.74 s,

whereas failure of a Raft follower produced gaps below 0.1 s in the evaluated three-replica configuration. EPaxos replica failure likewise produced short interruptions in the tested scenario. These results show that failure behaviour depends on which replica is removed and how the request-processing path is structured. The election experiment further characterised the Raft recovery process, with the measured number of failed election attempts strongly associated with the observed availability gap. These measurements describe the tested recovery configuration rather than protocol-independent recovery bounds.

The implementation-sensitivity experiments provide the strongest qualification to direct protocol comparisons. The evaluated Raft and EPaxos implementations exhibited substantially different absolute performance and different responses to injected communication cost, including substantial differences between implementations of the same protocol family. Persistence, request processing, transport handling, scheduling, and adapter behaviour can therefore materially affect measured throughput and latency without changing the underlying consensus algorithm. Absolute benchmark results should consequently be interpreted as properties of the evaluated implementation and environment rather than as intrinsic performance constants of Raft or EPaxos.

Taken together, the results do not establish a universal performance ordering between leader-based and leaderless consensus. Instead, they show that the observed relationship depends jointly on consensus structure, implementation, workload, replica count, client concurrency, communication conditions, and failure mode. The evaluated Raft configuration exposes the concentration and failure dependency associated with a leader-centred request path, while the evaluated EPaxos configuration exposes a different trade-off through distributed proposal placement and dependency coordination.

The study is limited by its single-host containerised environment, application-level rather than physical network-delay injection, tested cluster sizes, closed-loop workload model, and the specific implementations evaluated. The correctness experiments additionally establish state convergence under the tested workloads and failures rather than formal consensus correctness or general linearizability. These limitations restrict direct generalisation to production deployments and larger or geographically distributed systems.

Nevertheless, the experimental methodology provides a reproducible basis for studying consensus implementations under controlled changes in workload and system conditions. Future work can extend the comparison to additional independent implementations, larger clusters, multi-host deployments, network-level emulation, detailed profiling, and workloads that exercise stronger ordering and contention conditions. Such extensions would help further distinguish protocol-level effects from implementation-specific behaviour.

Acknowledgments

Generative AI tools were used during the preparation of this work for assistance with benchmark and laboratory code

development, language editing, proofreading, and drafting and revision of manuscript text. The author independently defined the research questions, experimental design, overall methodology, and interpretation of the study. Experimental execution, data analysis, and verification of the reported results were conducted and checked by the author. AI-generated suggestions were reviewed and revised by the author, who is responsible for the final content and technical claims of the manuscript.

References

- [1] etcd Authors. 2025. etcd: Distributed Reliable Key-Value Store (raft package). <https://github.com/etcd-io/etcd/tree/main/raft>. Version v3.7.0.
- [2] HashiCorp. 2025. HashiCorp Raft: A Raft Consensus Protocol Implementation in Go. <https://github.com/hashicorp/raft>. Version v1.7.3.
- [3] Heidi Howard and Richard Mortier. 2020. Paxos vs Raft: Have We Reached Consensus on Distributed Consensus?. In *7th Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC 20)*. ACM, 1–9. doi:10.1145/3380787.3393681
- [4] Leslie Lamport. 1998. The Part-Time Parliament. *ACM Transactions on Computer Systems* 16, 2 (1998), 133–169. doi:10.1145/279227.279229
- [5] Iulian Moraru, David G. Andersen, and Michael Kaminsky. 2013. There Is More Consensus in Egalitarian Parliaments. In *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP 13)*. ACM, Farmington, PA, 358–372. doi:10.1145/2517349.2522738
- [6] Diego Ongaro and John Ousterhout. 2014. In Search of an Understandable Consensus Algorithm. In *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC 14)*. USENIX Association, Philadelphia, PA, 305–319. <https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro>
- [7] Sarah Tollman, Seo Jin Park, and John Ousterhout. 2021. EPaxos Revisited. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*. USENIX Association, 613–632. <https://www.usenix.org/conference/nsdi21/presentation/tollman>
- [8] Nathan van Benschoten. 2025. EPaxos: Egalitarian Paxos in Go. <https://github.com/nvanbenschoten/epaxos>. Commit 425bd36.
- [9] Robbert van Renesse, Nicolas Schiper, and Fred B. Schneider. 2015. Vive La Différence: Paxos vs. Leaderless Replication. In *Proceedings of the 28th ACM Symposium on Principles of Distributed Computing (PODC 15)*. ACM, 242–251. doi:10.1145/2767386.2767419